

# UNIVERSIDAD DE GUADALAJARA



## CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

### Programación para Internet

#### Reporte de práctica

Nombre del alumno: Quijas Contreras Victor Manuel  
Código del alumno: 220429771  
Profesor: Erasmo Gabriel Martínez Soltero  
Título de la práctica: "Programación Orientada a Objetos"  
Fecha: 23 de agosto 2025

### 1.¿Qué es la Programación Orientado a Objetos?

Un paradigma en la programación donde se trabaja con clases y objetos. Las clases son plantillas que definen el comportamiento de los objetos, que son instancias de las clases.

### 2.¿Qué es una Clase?

Plantilla utilizada para definir los atributos (características o datos) y métodos (comportamiento o acciones) de uno o varios objetos.

### **3.¿Qué es un Objeto?**

Una instancia de una clase dentro del código, con un estado y comportamientos específicos.

### **4.¿Qué es un Atributo?**

Un dato, valor, variable que posee una clase.

### **5.¿Qué es Método?**

Una función definida dentro de esa clase, define el comportamiento del objeto y sus interacciones con los atributos.

### **6.¿Qué es un Constructor?**

Función especial encargada de prepara e inicializar los atributos de la clase, dejandola lista para crear las instancias de esta.

### **7.Diferencias entre atributos/métodos publicos, privados y protegidos**

- Público sirve acceder a ese método o atributo de una clase, desde cualquier punto del programa.
- Privado hace que solo los métodos de esa clase, tengan acceso a ellos.
- Protegido, similar al público, pero habilita la posibilidad de que clases heredadas tengan acceso a la información.

### **8.¿Qué es Herencia?**

Capacidad de las clases para poder acceder a métodos y atributos de otras clases. La herencia puede ordenarse de manera jerarquica, existir una clase padre, hijo y espiritu santo.

### **9.¿Qué es Polimorfismo?**

Capacidad de que varios objetos de diferentes clases se comporten como instancias de una misma clase base y que cada objeto responda de manera diferente a un mismo método. Hay dos tipos de Polimorfismo, en tiempo de ejecución y en tiempo de compilación.

## 10.¿Qué es un atributo o método estático?

Aquel que puede invocarse o acceder a él sin necesidad de crear la instancia del objeto.

## 11.Tres tips o buenas practicas para programar en Orientado a Objetos

1. Single Responsibility Principle (SRP): Crear clases especializadas en responsabilidades únicas a crear una sola clase con múltiples responsabilidades. 2. Open/Closed Principle (OCP): Crear métodos priorizando la compatibilidad de poder extenderlos y así para evitar su modificación a posterior.

3.Composición sobre herencia: Preferir construir objetos combinando componentes más simples en lugar de crear jerarquías de herencia profundas.

## 12.Screenshot de la definición de una clase con dos atributos y dos meto- dos

-Java Script:

```
class Persona {  
  // Constructor con dos atributos  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  
  // Primer método  
  saludar() {  
    return `Hola, soy ${this.nombre}`;  
  }  
  
  // Segundo método  
  esMayorDeEdad() {  
    return this.edad >= 18;  
  }  
}
```

-PHP:

```
<?php
class Persona {
    // Atributos
    public $nombre;
    public $edad;

    // Constructor con dos atributos
    public function __construct($nombre, $edad) {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }

    // Primer método
    public function saludar() {
        return "Hola, soy " . $this->nombre;
    }

    // Segundo método
    public function esMayorDeEdad() {
        return $this->edad >= 18;
    }
}
```

-Python:

```
class Persona:
    # Constructor con dos atributos
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    # Primer método
    def saludar(self):
        return f"Hola, soy {self.nombre}"

    # Segundo método
    def es_mayor_de_edad(self):
        return self.edad >= 18
```

## 13. Screenshot de la creación de un objeto

-Java Script:

```
const persona1 = new Persona("Ana", 25);
```

-PHP:

```
$persona1 = new Persona("Carlos", 30);
```

-Python:

```
persona1 = Persona("Maria", 20)
```

## 14. Screenshot de cómo se accede a un atributo público

-Java Script:

```
console.log(persona1.nombre); // "Ana" - Acceso directo  
console.log(persona1.edad);   // 25 - Acceso directo
```

-PHP:

```
echo $persona1->nombre; // "Carlos" - Acceso directo  
echo $persona1->edad;   // 30 - Acceso directo
```

-Python:

```
print(persona1.nombre) # "María" - Acceso directo  
print(persona1.edad)   # 20 - Acceso directo
```

## 15. Screenshot de cómo se manda llamar un método público

-Java Script:

```
console.log(persona1.saludar()); // "Hola, soy Ana"  
console.log(persona1.esMayorDeEdad()); // true
```

-PHP:

```
echo $persona1->saludar(); // "Hola, soy Carlos"  
echo $persona1->esMayorDeEdad() ? "true" : "false"; // true
```

-Python:

```
print(persona1.saludar()) # "Hola, soy María"  
print(persona1.es_mayor_de_edad()) # True
```

## Referencias

Se enumeran las fuentes bibliográficas utilizadas para la realización de la práctica.